Математическое моделирование

Лабораторная работа № 6

Абрикосов Артем

Содержание

1 Цель работы **6**

2 Задание **7**

3 Выполнение лабораторной работы **8**
3.1 Теоретические сведения 8

4 Эпидемиологическая модель SIR: численное решение и визуализация **11**
4.1 Инициализация проекта и загрузка пакетов 11
4.2 Начальные условия и временной интервал 12
4.3 Параметры модели 12
4.4 Первая модель 12
4.5 Вторая модель 13
4.6 Утилита: один прогон модели 13
4.7 Запуск первой модели 16
4.8 Запуск второй модели 16
4.9 Вывод первых строк таблиц 17
4.10 Показать график в интерактивной среде 17

5 Параметрическое исследование эпидемиологической модели SIR **18**
5.1 Активация проекта и загрузка пакетов 18
5.2 Установка каталогов 18
5.3 Определение моделей 19
5.4 Базовые параметры 19
5.5 Функция для запуска одного эксперимента 20
5.6 Запуск базового эксперимента: первая система 23
5.7 Визуализация базового эксперимента 24
5.8 Фазовый портрет I(S) 25
5.9 Второй базовый эксперимент 26
5.10 Параметрическое сканирование 28
5.11 Запуск всех экспериментов 29
5.12 Анализ результатов сканирования 31
5.13 Сравнительный график S(t) 32
5.14 Сравнительный график I(t) 33
5.15 Сравнительный график R(t) 34
5.16 Сравнительный фазовый портрет S-I 36

5.17 График зависимости I_max от параметров 37
5.18 График зависимости norm_final от параметров 38
5.19 Бенчмаркинг 40
5.20 График времени вычисления 42
5.21 Сохранение всех результатов 43
5.22 Базовые эксперименты 44
5.23 Параметрическое сканирование 48
5.24 Выводы 56

Список литературы **57**

Список иллюстраций

5.1 Первая модель: зависимость S(t), I(t), R(t) 44
5.2 Первая модель: фазовый портрет 45
5.3 Вторая модель: зависимость S(t), I(t), R(t) 46
5.4 Вторая модель: фазовый портрет 47
5.5 Сканирование траекторий S(t) 48
5.6 Сканирование траекторий I(t) 49
5.7 Сканирование траекторий R(t) 50
5.8 Сканирование фазовых траекторий 51
5.9 Зависимость norm_final 53
5.10 Зависимость I_{max} 54
5.11 Зависимость времени вычисления 55

Список таблиц

1 Цель работы

Изучить эпидемиологическую модель SIR и проанализировать особенности её динамики при различных начальных условиях.

2 Задание

1. Изучить математическую модель распространения эпидемии.
2. Построить графики изменения численности особей в каждой из трёх групп. Рассмотреть развитие эпидемии для двух случаев: $I(0) \leq I^*$ и $I(0) > I^*$.

3 Выполнение лабораторной работы

3.1 Теоретические сведения

Рассмотрим простейшую модель распространения инфекционного заболевания. Пусть имеется некоторая изолированная популяция, состоящая из N особей. В рамках модели всё население делится на три основные группы.

Первая группа включает восприимчивых к заболеванию, но ещё здоровых особей. Их численность обозначается через $S(t)$. Вторая группа состоит из инфицированных особей, которые одновременно являются носителями и распространителями инфекции. Их количество обозначим через $I(t)$. Третья группа описывает особей, которые уже выздоровели и приобрели иммунитет к заболеванию. Эта величина обозначается как $R(t)$.

До тех пор, пока число инфицированных не превышает некоторого критического уровня I^*, будем считать, что заболевшие изолированы и не передают инфекцию здоровым особям. Если же выполняется условие $I(t) > I^*$, то инфицированные начинают заражать восприимчивых к заболеванию особей.

Следовательно, скорость изменения числа восприимчивых особей $S(t)$ задаётся следующим образом:

$$\frac{dS}{dt} = \begin{cases} -\alpha S & \text{,если } I(t) > I^* \\ 0 & \text{,если } I(t) \leq I^* \end{cases}$$

Так как каждая восприимчивая особь после заражения переходит в группу

инфицированных, скорость изменения числа инфицированных определяется разностью между количеством новых заболевших и количеством тех, кто покидает группу инфицированных вследствие лечения. Поэтому для $I(t)$ получаем:

$$\frac{dI}{dt} = \begin{cases} \alpha S - \beta I & \text{,если } I(t) > I^* \\ -\beta I & \text{,если } I(t) \leq I^* \end{cases}$$

Скорость изменения числа выздоровевших особей, приобретающих иммунитет к болезни, имеет вид:

$$\frac{dR}{dt} = \beta I$$

Коэффициенты α и β являются постоянными пропорциональности. Параметр α характеризует интенсивность заражения, а параметр β описывает скорость выздоровления. Для однозначного определения решения системы необходимо задать начальные условия.

Будем считать, что в начальный момент времени $t = 0$ число особей с иммунитетом равно $R(0) = 0$, а начальные количества инфицированных и восприимчивых к заболеванию особей задаются величинами $I(0)$ и $S(0)$ соответственно. Для анализа развития эпидемии необходимо рассмотреть два режима: $I(0) \leq I^*$ и $I(0) > I^*$.

3.1.1 Задача

На некотором острове началась эпидемия. Известно, что общее число жителей острова равно $N = 11400$. В момент начала эпидемии, то есть при $t = 0$, число заболевших людей, являющихся распространителями инфекции, составляет $I(0) = 250$. Количество здоровых людей, уже имеющих иммунитет к заболеванию, равно $R(0) = 47$.

Следовательно, начальное число людей, восприимчивых к заболеванию, но

ещё не заражённых, определяется по формуле:

$$S(0) = N - I(0) - R(0)$$

Необходимо построить графики изменения численности особей в каждой из трёх групп.

Рассматриваются два варианта протекания эпидемии:

1. $I(0) \leq I^*$
2. $I(0) > I^*$

Для моделирования динамики системы и построения графиков использовались внешние файлы с программным кодом:

4 Эпидемиологическая модель SIR: численное решение и визуализация

Цель: решить две системы ОДУ для модели распространения заболевания, построить графики S(t), I(t), R(t), сохранить изображения и таблицы с результатами.

4.1 Инициализация проекта и загрузка пакетов

```
using DrWatson
@quickactivate "project"

using DifferentialEquations
using Plots
using DataFrames
using CSV
using JLD2

script_name = isempty(PROGRAM_FILE) ? "interactive" : splitext(basename(PROGRAM_FILE)
mkpath(plotsdir(script_name))
mkpath(datadir(script_name))
```

4.2 Начальные условия и временной интервал

```
N = 11400.0
I0 = 250.0
R0 = 47.0
S0 = N - I0 - R0

u0 = [S0, I0, R0]

t0 = 0.0
tmax = 200.0
tspan = (t0, tmax)
```

4.3 Параметры модели

```
a = 0.11
b = 0.02

p = (a = a, b = b)
```

4.4 Первая модель

S(t) — число восприимчивых I(t) — число инфицированных R(t) — число выбывших / выздоровевших

dS/dt = 0 dI/dt = b*I dR/dt = -b*I

```
function sir_case_1!(dy, y, p, t)
    dy[1] = 0.0
    dy[2] = p.b * y[2]
    dy[3] = -p.b * y[2]
end
```

4.5 Вторая модель

dS/dt = -aS $dI/dt = a$S - bI $dR/dt = b$I

```
function sir_case_2!(dy, y, p, t)
    dy[1] = -p.a * y[1]
    dy[2] =  p.a * y[1] - p.b * y[2]
    dy[3] =  p.b * y[2]
end
```

4.6 Утилита: один прогон модели

```
function run_case(case_name; model!, u0, tspan, p, nt = 1000)
```

— Сетка по времени —

```
    tgrid = collect(LinRange(tspan[1], tspan[2], nt))
```

— Решение ОДУ —

```
    prob = ODEProblem(model!, u0, tspan, p)
    sol = solve(prob, Tsit5(), saveat = tgrid)
```

— Таблица с результатами —

```
df = DataFrame(
    t = sol.t,
    S = [u[1] for u in sol.u],
    I = [u[2] for u in sol.u],
    R = [u[3] for u in sol.u]
)
```

— График S(t), I(t), R(t) —

```
plt_time = plot(
    sol,
    xlabel = "t",
    ylabel = "Численность",
    label = ["S(t) — восприимчивые" "I(t) — инфицированные" "R(t) — выбывшие"],
    lw = 2,
    title = "Динамика SIR — $case_name",
    legend = :topright
)
```

— Фазовый портрет I(S) —

```
plt_phase_si = plot(
    sol,
    idxs = (1, 2),
    xlabel = "S",
    ylabel = "I",
    lw = 2,
    label = "Фазовая траектория I(S)",
    title = "Фазовый портрет S-I — $case_name",
```

```
        legend = :topright
    )
```

— Фазовый портрет R(I) —

```
    plt_phase_ir = plot(
        sol,
        idxs = (2, 3),
        xlabel = "I",
        ylabel = "R",
        lw = 2,
        label = "Фазовая траектория R(I)",
        title = "Фазовый портрет I-R — $case_name",
        legend = :topright
    )
```

— Сохранение графиков —

```
    savefig(plt_time, plotsdir(script_name, "$(case_name)_time.png"))
    savefig(plt_phase_si, plotsdir(script_name, "$(case_name)_phase_SI.png"))
    savefig(plt_phase_ir, plotsdir(script_name, "$(case_name)_phase_IR.png"))
```

— Сохранение таблицы —

```
    CSV.write(datadir(script_name, "table_$(case_name).csv"), df)
```

— Сохранение данных в JLD2 —

```
    @save datadir(script_name, "data_$(case_name).jld2") df p u0 tspan nt

    return (
```

```
        sol = sol,
        df = df,
        plt_time = plt_time,
        plt_phase_si = plt_phase_si,
        plt_phase_ir = plt_phase_ir
    )
end
```

4.7 Запуск первой модели

```
res_1 = run_case(
    "sir_case_1";
    model! = sir_case_1!,
    u0 = u0,
    tspan = tspan,
    p = p,
    nt = 1000
)
```

4.8 Запуск второй модели

```
res_2 = run_case(
    "sir_case_2";
    model! = sir_case_2!,
    u0 = u0,
    tspan = tspan,
    p = p,
```

```
    nt = 1000
)
```

4.9 Вывод первых строк таблиц

```
println("Первые 5 строк таблицы результатов для первой модели:")
println(first(res_1.df, 5))

println()
println("Первые 5 строк таблицы результатов для второй модели:")
println(first(res_2.df, 5))
```

4.10 Показать график в интерактивной среде

```
res_1.plt_time
```

5 Параметрическое исследование эпидемиологической модели SIR

5.1 Активация проекта и загрузка пакетов

```
using DrWatson
@quickactivate "project"

using DifferentialEquations
using DataFrames
using Plots
using JLD2
using BenchmarkTools
using CSV
using Statistics
```

5.2 Установка каталогов

```
script_name = isempty(PROGRAM_FILE) ? "interactive" : splitext(basename(PROGRAM_FILE)
mkpath(plotsdir(script_name))
mkpath(datadir(script_name))
```

5.3 Определение моделей

Первая система: dS/dt = 0 dI/dt = b*I dR/dt = -b*I

```
function sir_case_1!(dy, y, p, t)
    dy[1] = 0.0
    dy[2] = p.b * y[2]
    dy[3] = -p.b * y[2]
end
```

Вторая система: dS/dt = -a*S dI/dt = a*S - b*I dR/dt = b*I

```
function sir_case_2!(dy, y, p, t)
    dy[1] = -p.a * y[1]
    dy[2] =  p.a * y[1] - p.b * y[2]
    dy[3] =  p.b * y[2]
end
```

5.4 Базовые параметры

```
base_params = Dict(
    :N => 11400.0,
    :I0 => 250.0,
    :R0 => 47.0,

    :tspan => (0.0, 200.0),
    :nt => 1000,

    :model_type => :case1,     # :case1 или :case2
```

```
    :a => 0.11,
    :b => 0.02,

    :solver => Tsit5(),
    :experiment_name => "sir_base_experiment"
)

println("Базовые параметры эксперимента:")
for (key, value) in base_params
    println(" $key = $value")
end
```

5.5 Функция для запуска одного эксперимента

```
function run_single_experiment(params::Dict)
    N = params[:N]
    I0 = params[:I0]
    R0 = params[:R0]
    S0 = N - I0 - R0

    tspan = params[:tspan]
    nt = params[:nt]
    model_type = params[:model_type]

    a = params[:a]
    b = params[:b]
```

```
    solver = params[:solver]

    u0 = [S0, I0, R0]
    tgrid = collect(LinRange(tspan[1], tspan[2], nt))

    p = (a = a, b = b)

    if model_type == :case1
        prob = ODEProblem(sir_case_1!, u0, tspan, p)
    else
        prob = ODEProblem(sir_case_2!, u0, tspan, p)
    end

    sol = solve(prob, solver; saveat = tgrid)

    S_vals = [u[1] for u in sol.u]
    I_vals = [u[2] for u in sol.u]
    R_vals = [u[3] for u in sol.u]
```

— Мини-анализ —

```
    S_final = S_vals[end]
    I_final = I_vals[end]
    R_final = R_vals[end]

    S_max = maximum(S_vals)
    I_max = maximum(I_vals)
    R_max = maximum(R_vals)
```

```
    S_min = minimum(S_vals)
    I_min = minimum(I_vals)
    R_min = minimum(R_vals)

    S_mean = mean(S_vals)
    I_mean = mean(I_vals)
    R_mean = mean(R_vals)

    total_final = S_final + I_final + R_final
    norm_final = sqrt(S_final^2 + I_final^2 + R_final^2)

    return Dict(
        "solution" => sol,
        "t_points" => sol.t,

        "S_values" => S_vals,
        "I_values" => I_vals,
        "R_values" => R_vals,

        "S_final" => S_final,
        "I_final" => I_final,
        "R_final" => R_final,

        "S_max" => S_max,
        "I_max" => I_max,
        "R_max" => R_max,

        "S_min" => S_min,
```

```
        "I_min" => I_min,
        "R_min" => R_min,

        "S_mean" => S_mean,
        "I_mean" => I_mean,
        "R_mean" => R_mean,

        "total_final" => total_final,
        "norm_final" => norm_final,

        "parameters" => params
    )
end
```

5.6 Запуск базового эксперимента: первая система

```
data_base, path_base = produce_or_load(
    datadir(script_name, "single"),
    base_params,
    run_single_experiment;
    prefix = "sir",
    tag = false,
    verbose = true
)

println("\nРезультаты базового эксперимента:")
println(" S_final: ", data_base["S_final"])
```

```
println(" I_final: ", data_base["I_final"])
println(" R_final: ", data_base["R_final"])
println(" I_max: ", data_base["I_max"])
println(" total_final: ", data_base["total_final"])
println(" norm_final: ", round(data_base["norm_final"]; digits = 4))
println(" Файл результатов: ", path_base)
```

5.7 Визуализация базового эксперимента

```
p1 = plot(
    data_base["t_points"], data_base["S_values"],
    label = "S(t) — восприимчивые",
    xlabel = "t",
    ylabel = "Численность",
    title = "Базовый эксперимент: model_type=$(base_params[:model_type])",
    lw = 2,
    legend = :topright,
    grid = true
)

plot!(
    p1,
    data_base["t_points"], data_base["I_values"],
    label = "I(t) — инфицированные",
    lw = 2
)
```

```
plot!(
    p1,
    data_base["t_points"], data_base["R_values"],
    label = "R(t) — выбывшие",
    lw = 2
)

savefig(p1, plotsdir(script_name, "single_experiment_case1.png"))
```

5.8 Фазовый портрет I(S)

```
p2 = plot(
    data_base["S_values"], data_base["I_values"],
    xlabel = "S",
    ylabel = "I",
    label = "Фазовая траектория I(S)",
    title = "Фазовый портрет S-I: model_type=$(base_params[:model_type])",
    lw = 2,
    legend = :topright,
    grid = true
)

savefig(p2, plotsdir(script_name, "single_experiment_case1_phase_SI.png"))
```

5.9 Второй базовый эксперимент

```
base_params2 = copy(base_params)
base_params2[:model_type] = :case2
base_params2[:experiment_name] = "sir_base_experiment_case2"

data_base2, path_base2 = produce_or_load(
    datadir(script_name, "single"),
    base_params2,
    run_single_experiment;
    prefix = "sir",
    tag = false,
    verbose = true
)

println("\nРезультаты второго базового эксперимента:")
println(" S_final: ", data_base2["S_final"])
println(" I_final: ", data_base2["I_final"])
println(" R_final: ", data_base2["R_final"])
println(" I_max: ", data_base2["I_max"])
println(" total_final: ", data_base2["total_final"])
println(" norm_final: ", round(data_base2["norm_final"]; digits = 4))
println(" Файл результатов: ", path_base2)

p3 = plot(
    data_base2["t_points"], data_base2["S_values"],
    label = "S(t) — восприимчивые",
    xlabel = "t",
```

```
    ylabel = "Численность",
    title = "Базовый эксперимент: model_type=$(base_params2[:model_type])",
    lw = 2,
    legend = :topright,
    grid = true
)

plot!(
    p3,
    data_base2["t_points"], data_base2["I_values"],
    label = "I(t) — инфицированные",
    lw = 2
)

plot!(
    p3,
    data_base2["t_points"], data_base2["R_values"],
    label = "R(t) — выбывшие",
    lw = 2
)

savefig(p3, plotsdir(script_name, "single_experiment_case2.png"))

p4 = plot(
    data_base2["S_values"], data_base2["I_values"],
    xlabel = "S",
    ylabel = "I",
    label = "Фазовая траектория I(S)",
```

```
    title = "Фазовый портрет S-I: model_type=$(base_params2[:model_type])",
    lw = 2,
    legend = :topright,
    grid = true
)

savefig(p4, plotsdir(script_name, "single_experiment_case2_phase_SI.png"))
```

5.10 Параметрическое сканирование

```
param_grid = Dict(
    :N => [11400.0],
    :I0 => [250.0],
    :R0 => [47.0],

    :tspan => [(0.0, 200.0)],
    :nt => [1000],

    :model_type => [:case1, :case2],

    :a => [0.05, 0.08, 0.11, 0.15],
    :b => [0.01, 0.02, 0.03, 0.05],

    :solver => [Tsit5()],
    :experiment_name => ["sir_parametric_scan"]
)
```

```
all_params = dict_list(param_grid)

println("\n" * "="^60)
println("ПАРАМЕТРИЧЕСКОЕ СКАНИРОВАНИЕ")
println("Всего комбинаций параметров: ", length(all_params))
println("Исследуемые model_type: ", param_grid[:model_type])
println("Исследуемые a: ", param_grid[:a])
println("Исследуемые b: ", param_grid[:b])
println("="^60)
```

5.11 Запуск всех экспериментов

```
all_results = []
all_dfs = []

for (i, params) in enumerate(all_params)
    println(
        "Прогресс: $i/$(length(all_params)) | ",
        "model_type=$(params[:model_type]) | ",
        "a=$(params[:a]) | b=$(params[:b])"
    )

    data, path = produce_or_load(
        datadir(script_name, "parametric_scan"),
        params,
        run_single_experiment;
        prefix = "scan",
```

```
        tag = false,
        verbose = false
    )

    result_summary = merge(
        params,
        Dict(
            :S_final => data["S_final"],
            :I_final => data["I_final"],
            :R_final => data["R_final"],

            :S_max => data["S_max"],
            :I_max => data["I_max"],
            :R_max => data["R_max"],

            :S_min => data["S_min"],
            :I_min => data["I_min"],
            :R_min => data["R_min"],

            :S_mean => data["S_mean"],
            :I_mean => data["I_mean"],
            :R_mean => data["R_mean"],

            :total_final => data["total_final"],
            :norm_final => data["norm_final"],
            :filepath => path
        )
    )
```

```
    push!(all_results, result_summary)

    df = DataFrame(
        t = data["t_points"],
        S = data["S_values"],
        I = data["I_values"],
        R = data["R_values"],
        model_type = fill(string(params[:model_type]), length(data["t_points"])),
        a = fill(params[:a], length(data["t_points"])),
        b = fill(params[:b], length(data["t_points"]))
    )

    push!(all_dfs, df)
end
```

5.12 Анализ результатов сканирования

```
results_df = DataFrame(all_results)
full_timeseries_df = vcat(all_dfs...)

println("\nСводная таблица результатов:")
println(first(results_df, 10))

CSV.write(datadir(script_name, "results_summary.csv"), results_df)
CSV.write(datadir(script_name, "timeseries_full.csv"), full_timeseries_df)
```

5.13 Сравнительный график S(t)

```
p5 = plot(size = (950, 520), dpi = 150)

for params in all_params
    data, _ = produce_or_load(
        datadir(script_name, "parametric_scan"),
        params,
        run_single_experiment;
        prefix = "scan",
        tag = false,
        verbose = false
    )

    label_text = "$(params[:model_type]), a=$(params[:a]), b=$(params[:b])"

    plot!(
        p5,
        data["t_points"],
        data["S_values"],
        label = label_text,
        lw = 2,
        alpha = 0.8
    )
end

plot!(
    p5,
```

```
    xlabel = "t",
    ylabel = "S(t)",
    title = "Сканирование: траектории S(t)",
    legend = :outerright,
    grid = true
)

savefig(p5, plotsdir(script_name, "parametric_scan_S_comparison.png"))
```

5.14 Сравнительный график I(t)

```
p6 = plot(size = (950, 520), dpi = 150)

for params in all_params
    data, _ = produce_or_load(
        datadir(script_name, "parametric_scan"),
        params,
        run_single_experiment;
        prefix = "scan",
        tag = false,
        verbose = false
    )

    label_text = "$(params[:model_type]), a=$(params[:a]), b=$(params[:b])"

    plot!(
        p6,
```

```
        data["t_points"],
        data["I_values"],
        label = label_text,
        lw = 2,
        alpha = 0.8
    )
end

plot!(
    p6,
    xlabel = "t",
    ylabel = "I(t)",
    title = "Сканирование: траектории I(t)",
    legend = :outerright,
    grid = true
)

savefig(p6, plotsdir(script_name, "parametric_scan_I_comparison.png"))
```

5.15 Сравнительный график R(t)

```
p7 = plot(size = (950, 520), dpi = 150)

for params in all_params
    data, _ = produce_or_load(
        datadir(script_name, "parametric_scan"),
        params,
```

```
        run_single_experiment;
        prefix = "scan",
        tag = false,
        verbose = false
    )

    label_text = "$(params[:model_type]), a=$(params[:a]), b=$(params[:b])"

    plot!(
        p7,
        data["t_points"],
        data["R_values"],
        label = label_text,
        lw = 2,
        alpha = 0.8
    )
end

plot!(
    p7,
    xlabel = "t",
    ylabel = "R(t)",
    title = "Сканирование: траектории R(t)",
    legend = :outerright,
    grid = true
)

savefig(p7, plotsdir(script_name, "parametric_scan_R_comparison.png"))
```

5.16 Сравнительный фазовый портрет S-I

```
p8 = plot(size = (950, 520), dpi = 150)

for params in all_params
    data, _ = produce_or_load(
        datadir(script_name, "parametric_scan"),
        params,
        run_single_experiment;
        prefix = "scan",
        tag = false,
        verbose = false
    )

    label_text = "$(params[:model_type]), a=$(params[:a]), b=$(params[:b])"

    plot!(
        p8,
        data["S_values"],
        data["I_values"],
        label = label_text,
        lw = 2,
        alpha = 0.8
    )
end

plot!(
    p8,
```

```
    xlabel = "S",
    ylabel = "I",
    title = "Сканирование: фазовые траектории S-I",
    legend = :outerright,
    grid = true
)

savefig(p8, plotsdir(script_name, "parametric_scan_phase_SI_comparison.png"))
```

5.17 График зависимости I_max от параметров

```
p9 = plot(size = (900, 520), dpi = 150)

case1_sub = results_df[results_df.model_type .== :case1, :]
case2_sub = results_df[results_df.model_type .== :case2, :]

if nrow(case1_sub) > 0
    plot!(
        p9,
        case1_sub.b,
        case1_sub.I_max,
        seriestype = :scatter,
        label = "case1: I_max от b"
    )
end

if nrow(case2_sub) > 0
```

```
    plot!(
        p9,
        case2_sub.a,
        case2_sub.I_max,
        seriestype = :scatter,
        label = "case2: I_max от a"
    )
end

plot!(
    p9,
    xlabel = "Параметр модели",
    ylabel = "I_max",
    title = "Зависимость максимального числа инфицированных от параметров",
    legend = :topleft,
    grid = true
)

savefig(p9, plotsdir(script_name, "I_max_vs_parameter.png"))
```

5.18 График зависимости norm_final от параметров

```
p10 = plot(size = (900, 520), dpi = 150)

if nrow(case1_sub) > 0
    plot!(
        p10,
```

```
        case1_sub.b,
        case1_sub.norm_final,
        seriestype = :scatter,
        label = "case1: norm_final от b"
    )
end

if nrow(case2_sub) > 0
    plot!(
        p10,
        case2_sub.a,
        case2_sub.norm_final,
        seriestype = :scatter,
        label = "case2: norm_final от a"
    )
end

plot!(
    p10,
    xlabel = "Параметр модели",
    ylabel = "norm_final",
    title = "Зависимость norm_final от параметров модели",
    legend = :topleft,
    grid = true
)

savefig(p10, plotsdir(script_name, "norm_final_vs_parameter.png"))
```

5.19 Бенчмаркинг

```
println("\n" * "="^60)
println("БЕНЧМАРКИНГ ДЛЯ РАЗНЫХ ПАРАМЕТРОВ")
println("="^60)

benchmark_results = []

for params in all_params
    function benchmark_run()
        N = params[:N]
        I0 = params[:I0]
        R0 = params[:R0]
        S0 = N - I0 - R0

        u0 = [S0, I0, R0]
        p = (a = params[:a], b = params[:b])

        if params[:model_type] == :case1
            prob = ODEProblem(sir_case_1!, u0, params[:tspan], p)
        else
            prob = ODEProblem(sir_case_2!, u0, params[:tspan], p)
        end

        return solve(
            prob,
            params[:solver];
            saveat = LinRange(params[:tspan][1], params[:tspan][2], params[:nt])
```

```
        )
    end

    println(
        "\nБенчмарк для model_type=$(params[:model_type]), ",
        "a=$(params[:a]), b=$(params[:b]):"
    )

    bmark = @benchmark $benchmark_run() samples = 80 evals = 1
    tsec = median(bmark).time / 1e9

    println(" Медианное время: ", round(tsec; digits = 6), " сек")

    push!(
        benchmark_results,
        (
            model_type = string(params[:model_type]),
            a = params[:a],
            b = params[:b],
            time = tsec
        )
    )
end

bench_df = DataFrame(benchmark_results)
CSV.write(datadir(script_name, "benchmark_results.csv"), bench_df)
```

5.20 График времени вычисления

```
p11 = plot(size = (900, 520), dpi = 150)

bench_case1 = bench_df[bench_df.model_type .== "case1", :]
bench_case2 = bench_df[bench_df.model_type .== "case2", :]

if nrow(bench_case1) > 0
    plot!(
        p11,
        bench_case1.b,
        bench_case1.time,
        seriestype = :scatter,
        label = "case1: время от b"
    )
end

if nrow(bench_case2) > 0
    plot!(
        p11,
        bench_case2.a,
        bench_case2.time,
        seriestype = :scatter,
        label = "case2: время от a"
    )
end

plot!(
```

```
    p11,
    xlabel = "Параметр модели",
    ylabel = "Время вычисления, сек",
    title = "Зависимость времени решения ODE от параметров",
    legend = :topleft,
    grid = true
)

savefig(p11, plotsdir(script_name, "computation_time_vs_parameter.png"))
```

5.21 Сохранение всех результатов

```
@save datadir(script_name, "all_results.jld2") base_params base_params2 param_grid al
@save datadir(script_name, "all_plots.jld2") p1 p2 p3 p4 p5 p6 p7 p8 p9 p10 p11

println("\n" * "="^60)
println("ЛАБОРАТОРНАЯ РАБОТА ЗАВЕРШЕНА")
println("="^60)

println("\nРезультаты сохранены в:")
println(" • data/$(script_name)/single/ - базовые эксперименты")
println(" • data/$(script_name)/parametric_scan/ - параметрическое сканирование")
println(" • data/$(script_name)/results_summary.csv - сводная таблица")
println(" • data/$(script_name)/timeseries_full.csv - полные временные ряды")
println(" • data/$(script_name)/benchmark_results.csv - результаты бенчмаркинга")
println(" • data/$(script_name)/all_results.jld2 - сводные данные")
println(" • plots/$(script_name)/ - все графики")
```

```
println(" • data/$(script_name)/all_plots.jld2 - объекты графиков")
```

5.22 Базовые эксперименты

5.22.1 Первая модель (model_type = case1)

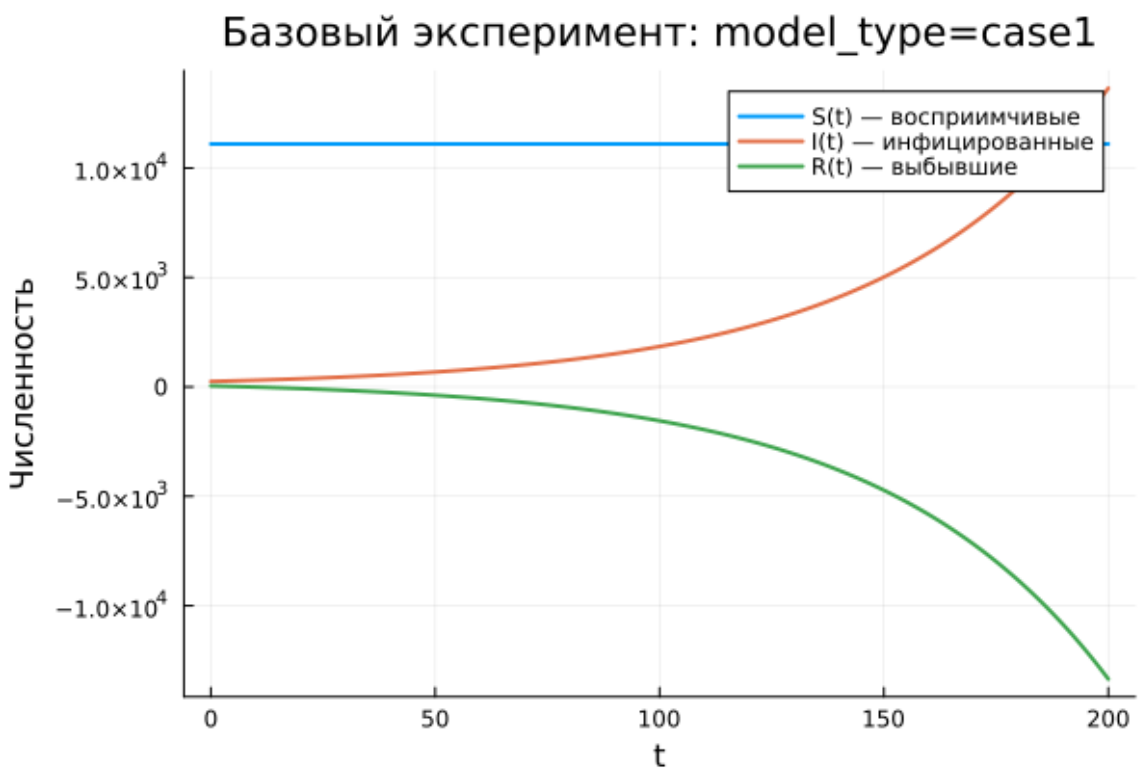

Рисунок 5.1: Первая модель: зависимость S(t), I(t), R(t)

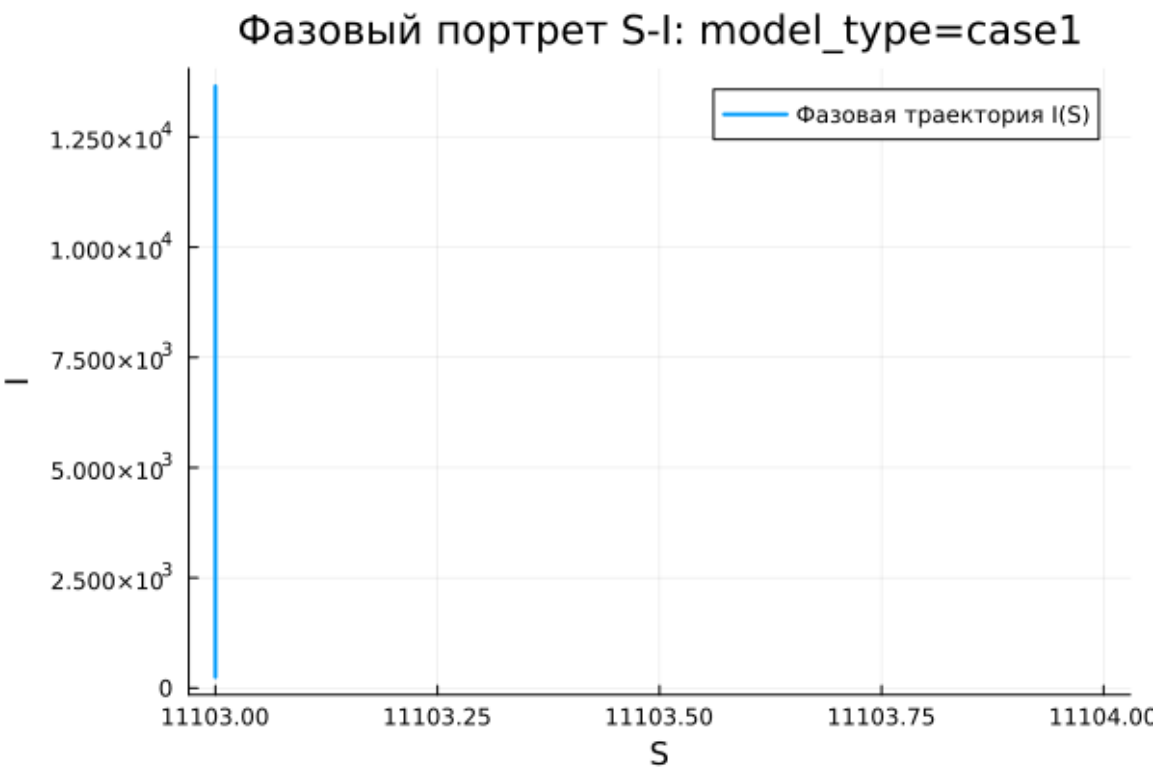

Рисунок 5.2: Первая модель: фазовый портрет

В первой модели система демонстрирует нетипичную динамику. Число восприимчивых особей $S(t)$ не изменяется с течением времени, что соответствует условию $dS/dt = 0$. При этом количество инфицированных $I(t)$ возрастает по экспоненциальному закону, а величина $R(t)$ уменьшается и со временем принимает отрицательные значения.

Такой результат не согласуется с классическим смыслом эпидемиологической модели. В корректной модели число выздоровевших не должно становиться отрицательным, а рост инфицированных должен ограничиваться конечным размером популяции. Полученная динамика связана с особенностями заданной системы уравнений: в ней отсутствует механизм, который ограничивал бы увеличение числа инфицированных, а переменная $R(t)$ теряет физическую интерпретацию.

Фазовый портрет также подтверждает аномальный характер поведения

системы. Поскольку $S(t)$ остаётся постоянным, траектория фактически превращается в вертикальную линию. Изменение происходит только вдоль оси I, поэтому двумерная динамика системы оказывается вырожденной.

Таким образом, первая модель описывает неограниченное увеличение числа инфицированных без последующей стабилизации. По этой причине её нельзя считать адекватным описанием реального распространения заболевания.

5.22.2 Вторая модель (model_type = case2)

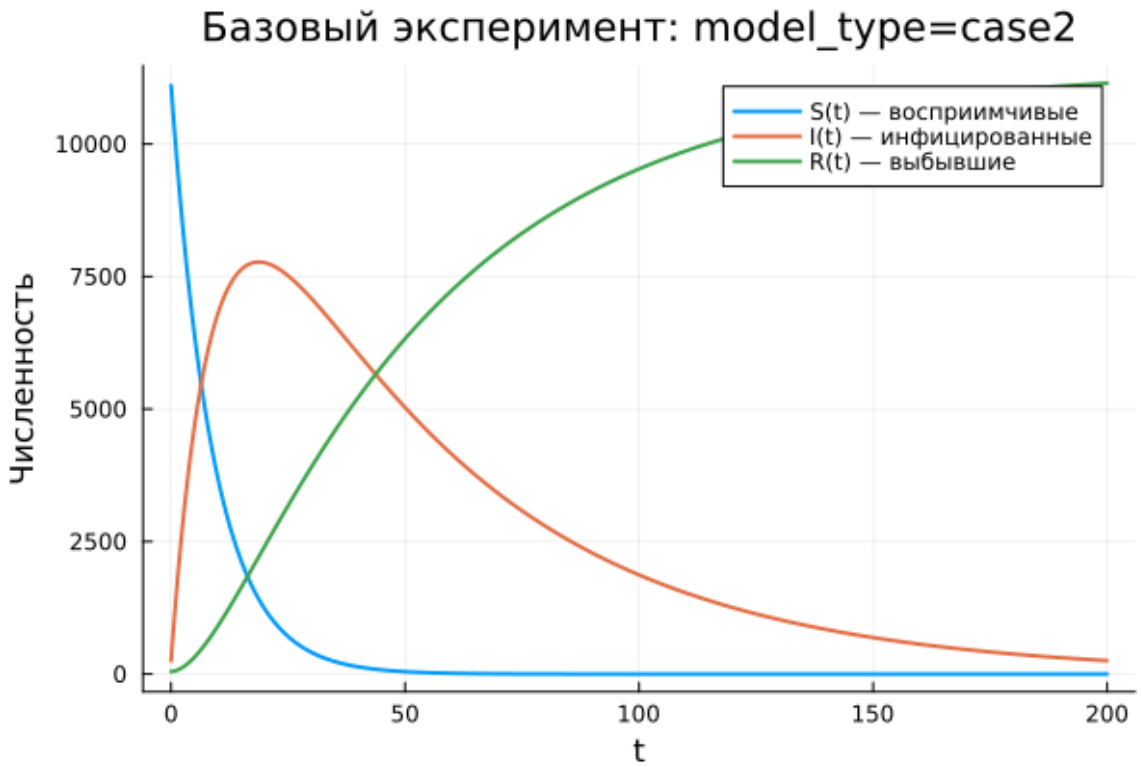

Рисунок 5.3: Вторая модель: зависимость S(t), I(t), R(t)

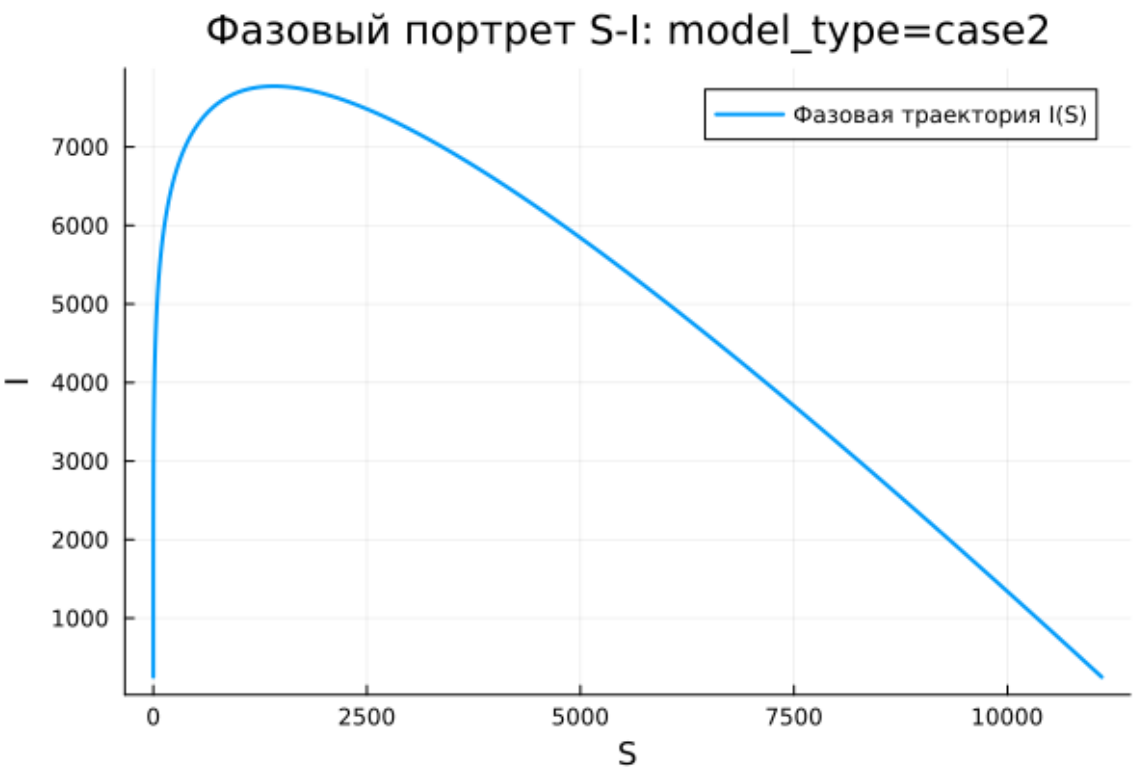

Рисунок 5.4: Вторая модель: фазовый портрет

Во второй модели наблюдается поведение, характерное для системы типа SIR. Число восприимчивых особей $S(t)$ постепенно уменьшается, что отражает процесс заражения. Количество инфицированных $I(t)$ сначала возрастает, затем достигает максимального значения, после чего начинает снижаться и стремится к нулю. В то же время число выбывших или выздоровевших особей $R(t)$ монотонно увеличивается.

Такая динамика соответствует естественному ходу эпидемического процесса в популяции конечного размера. В начальный период заболевание активно распространяется, поскольку восприимчивых особей достаточно много. Затем их число уменьшается, интенсивность заражения падает, и эпидемия постепенно затухает.

Фазовый портрет имеет вид незамкнутой кривой. Сначала траектория поднимается вверх, что соответствует росту $I(t)$ при одновременном уменьшении

$S(t)$. Затем она начинает спускаться вниз, отражая спад числа инфицированных после прохождения пика эпидемии.

В отличие от первой модели, здесь система стремится к стационарному режиму: $I(t) \to 0$, значение $S(t)$ выходит на некоторый остаточный уровень, а $R(t)$ достигает конечной величины.

5.23 Параметрическое сканирование

5.23.1 Траектории $S(t)$ для различных параметров

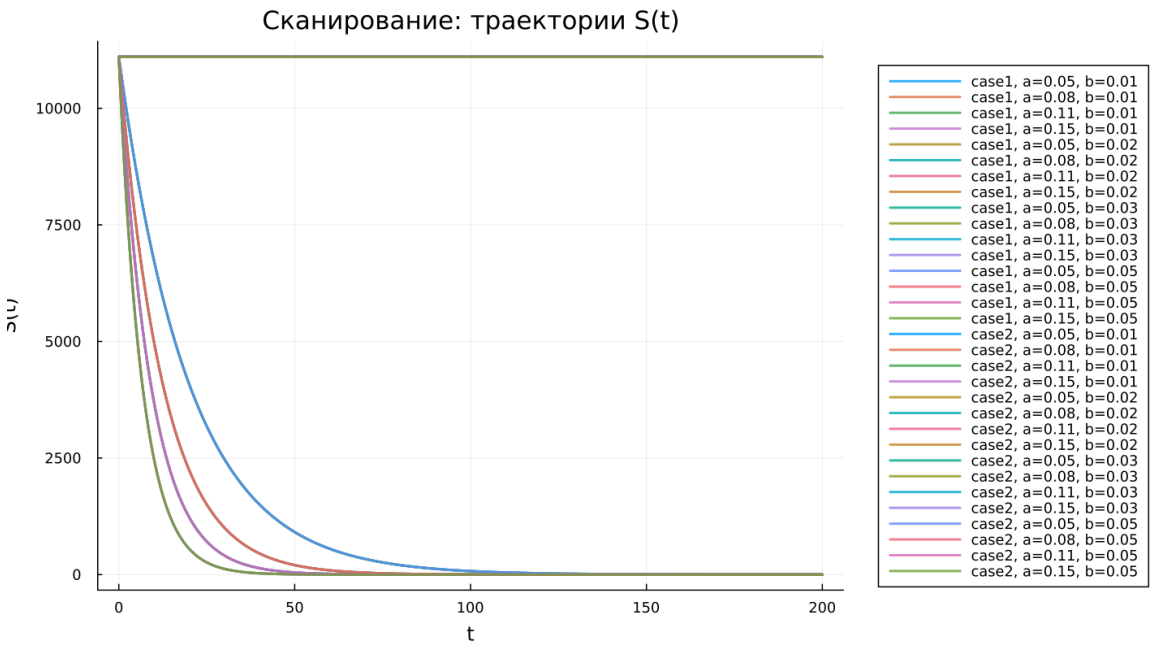

Рисунок 5.5: Сканирование траекторий S(t)

Анализ графиков $S(t)$ показывает, что две модели по-разному реагируют на изменение параметров. В первой модели, соответствующей случаю case1, величина $S(t)$ остаётся неизменной для всех рассматриваемых наборов параметров. Это напрямую связано с уравнением $dS/dt = 0$, поэтому изменение коэффициентов не оказывает влияния на динамику восприимчивых особей.

Во второй модели, соответствующей случаю case2, число восприимчивых убывает. Скорость этого убывания существенно зависит от параметра a. Чем больше значение a, тем быстрее уменьшается количество восприимчивых особей, что означает более интенсивное распространение инфекции.

Основные результаты наблюдений:

- в первой модели величина $S(t)$ остаётся постоянной;
- во второй модели число восприимчивых особей уменьшается;
- параметр a определяет скорость снижения $S(t)$.

5.23.2 Траектории $I(t)$ для различных параметров

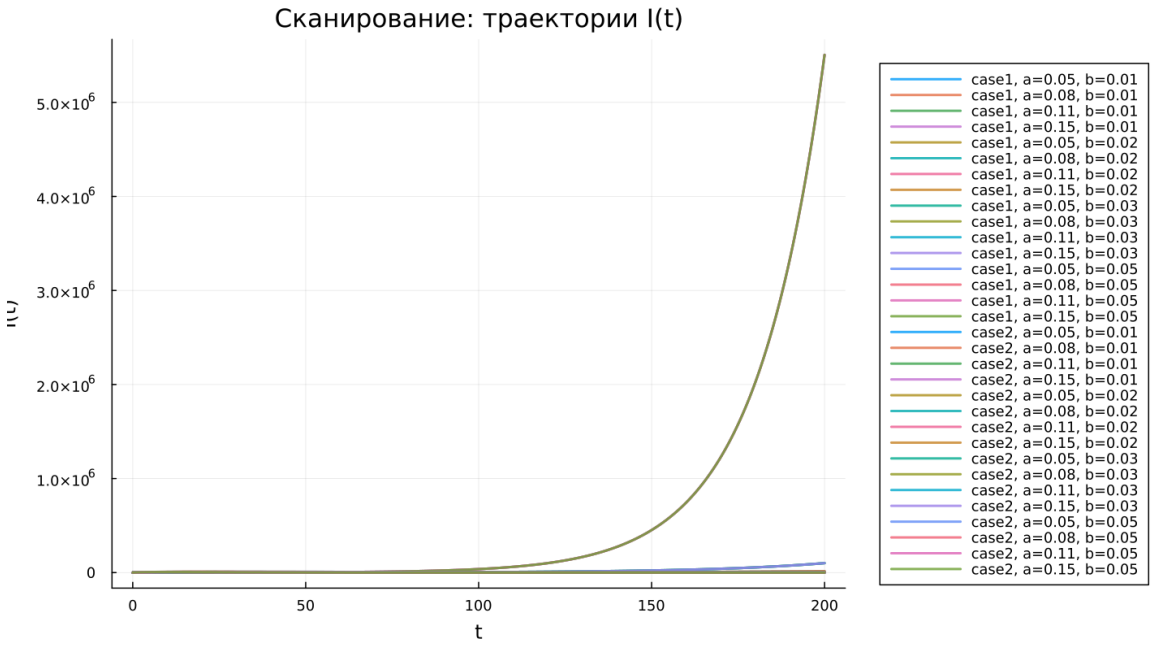

Рисунок 5.6: Сканирование траекторий I(t)

Для первой модели характерен экспоненциальный рост числа инфицированных $I(t)$ при всех выбранных значениях параметров. При увеличении параметра b скорость роста становится значительно выше, из-за чего значения $I(t)$ быстро достигают очень больших величин.

Во второй модели поведение инфицированных имеет иной характер. В начале процесса $I(t)$ увеличивается, затем достигает пика, после чего постепенно уменьшается. Параметр a влияет как на высоту максимума, так и на момент его достижения. При больших значениях a пик наступает раньше, а дальнейшее снижение происходит быстрее.

Можно выделить следующие особенности:

- первая модель приводит к неограниченному росту числа инфицированных;
- во второй модели формируется типичная эпидемическая волна;
- параметры системы определяют скорость и масштаб распространения заболевания.

5.23.3 Траектории $R(t)$ для различных параметров

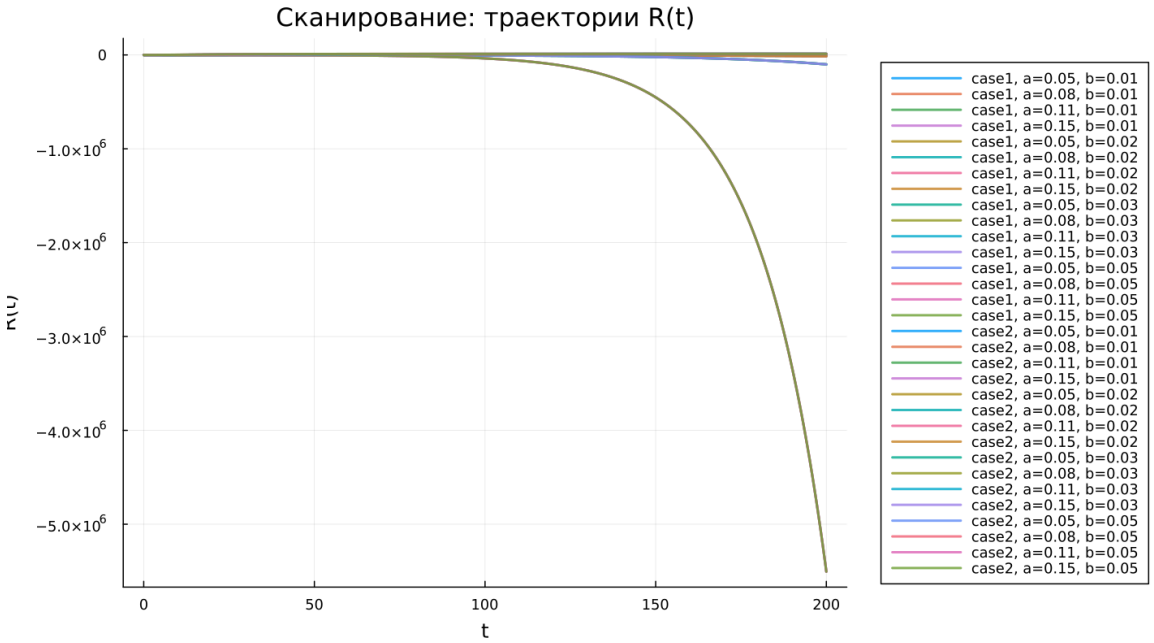

Рисунок 5.7: Сканирование траекторий R(t)

В первой модели величина $R(t)$ уменьшается и со временем становится отрицательной. По модулю значения $R(t)$ быстро возрастают, что указывает на

некорректность данной модели. Переменная, отвечающая за выздоровевших или выбывших особей, в таком случае теряет физический смысл.

Во второй модели ситуация принципиально иная. Значение $R(t)$ монотонно возрастает и стремится к некоторому предельному уровню. Это соответствует накоплению особей, которые переболели и больше не участвуют в распространении инфекции. Параметры модели влияют на скорость роста $R(t)$: при более интенсивном заражении накопление выздоровевших происходит быстрее.

Следовательно:

- первая модель даёт нефизичные значения $R(t)$;
- во второй модели наблюдается естественное увеличение числа выздоровевших;
- параметры влияют на скорость выхода $R(t)$ на предельное значение.

5.23.4 Фазовые траектории для различных параметров

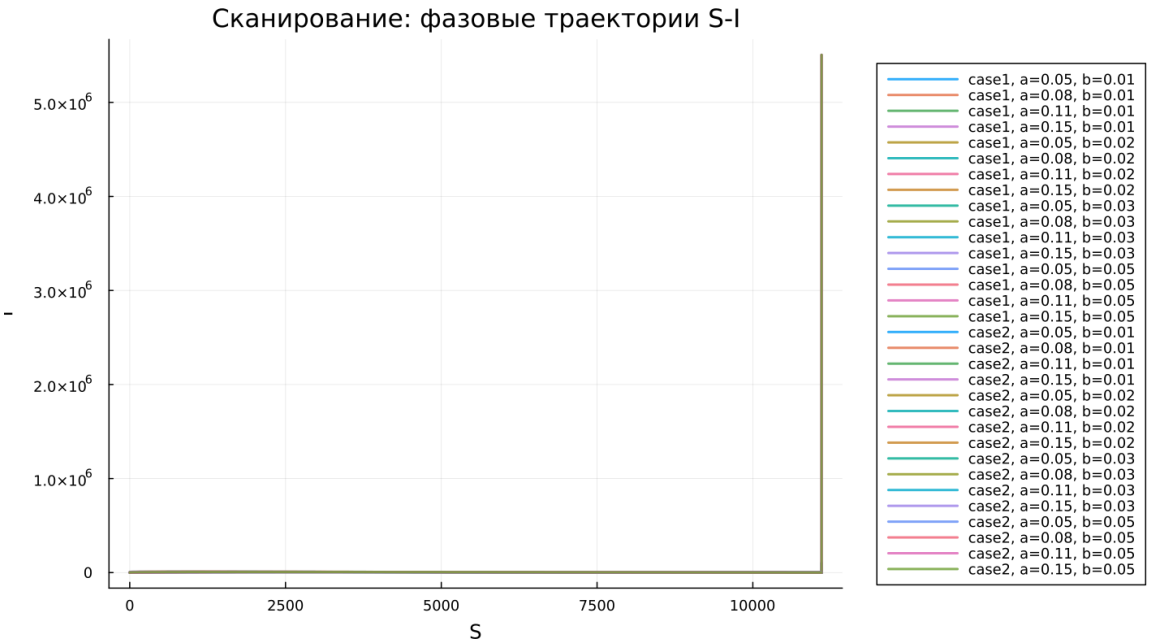

Рисунок 5.8: Сканирование фазовых траекторий

Фазовые портреты позволяют наглядно увидеть качественное различие между двумя моделями.

В первой модели все траектории имеют вид вертикальных линий, поскольку переменная S остаётся постоянной. Это показывает, что система фактически не обладает полноценной двумерной динамикой в фазовой плоскости (S, I).

Во второй модели фазовые траектории представляют собой характерные кривые, соответствующие развитию эпидемии. Сначала наблюдается рост числа инфицированных при уменьшении количества восприимчивых, затем после достижения максимума $I(t)$ начинается спад инфицированных при дальнейшем снижении $S(t)$.

Таким образом, изменение параметров не меняет общего характера поведения моделей:

- первая модель остаётся вырожденной;
- вторая модель сохраняет реалистичную структуру эпидемической динамики.

5.23.5 Анализ метрики norm_final

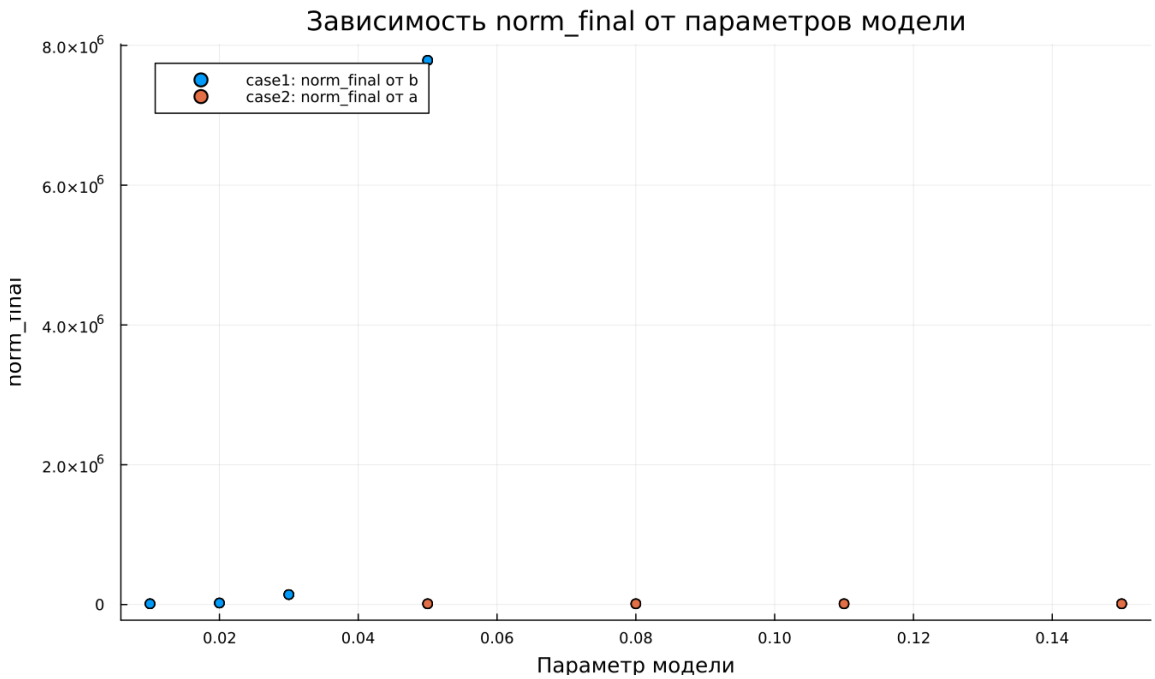

Рисунок 5.9: Зависимость norm_final

В работе рассматривалась метрика:

$$\text{norm_final} = \sqrt{S(t_{final})^2 + I(t_{final})^2 + R(t_{final})^2}.$$

Для первой модели значение norm_final быстро увеличивается при росте параметра b. Это объясняется тем, что $I(t)$ возрастает экспоненциально, а $R(t)$ принимает большие по модулю отрицательные значения. В результате итоговая норма становится очень большой.

Во второй модели значения данной метрики заметно меньше и изменяются более плавно. Это связано с тем, что система выходит на стационарное состояние: $I(t) \to 0$, а величины $S(t)$ и $R(t)$ остаются конечными.

Итоговые наблюдения:

- в первой модели метрика возрастает без ограничения;
- во второй модели она характеризует конечное состояние системы.

5.23.6 Анализ максимального числа инфицированных

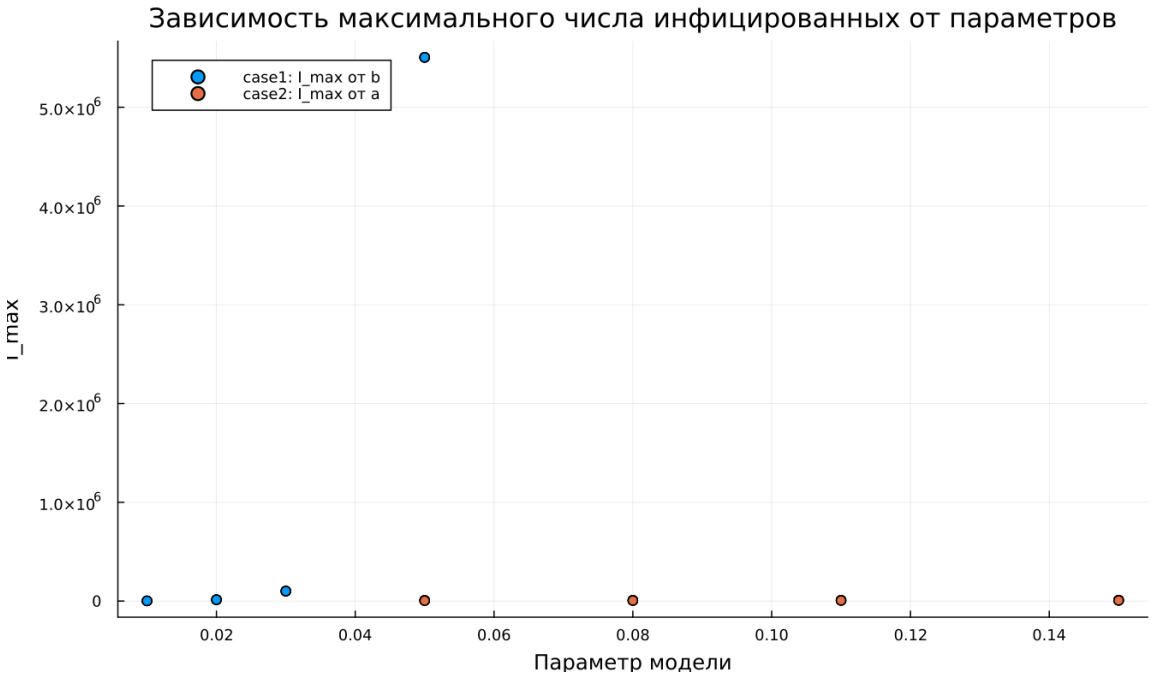

Рисунок 5.10: Зависимость I_{max}

Максимальное число инфицированных I_{max} существенно зависит от параметров модели.

В первой модели величина I_{max} достигает крайне больших значений. Это связано с тем, что в системе отсутствует механизм, ограничивающий рост числа инфицированных. При увеличении параметра b максимум резко возрастает.

Во второй модели значение I_{max} остаётся конечным и определяется параметром a. При увеличении a пик инфицированных достигается быстрее, однако его величина ограничивается конечной численностью популяции и дальнейшим уменьшением числа восприимчивых особей.

Следовательно:

- первая модель приводит к практически неограниченному увеличению I_{max};
- вторая модель описывает ограниченную эпидемическую волну.

5.23.7 Время вычислений

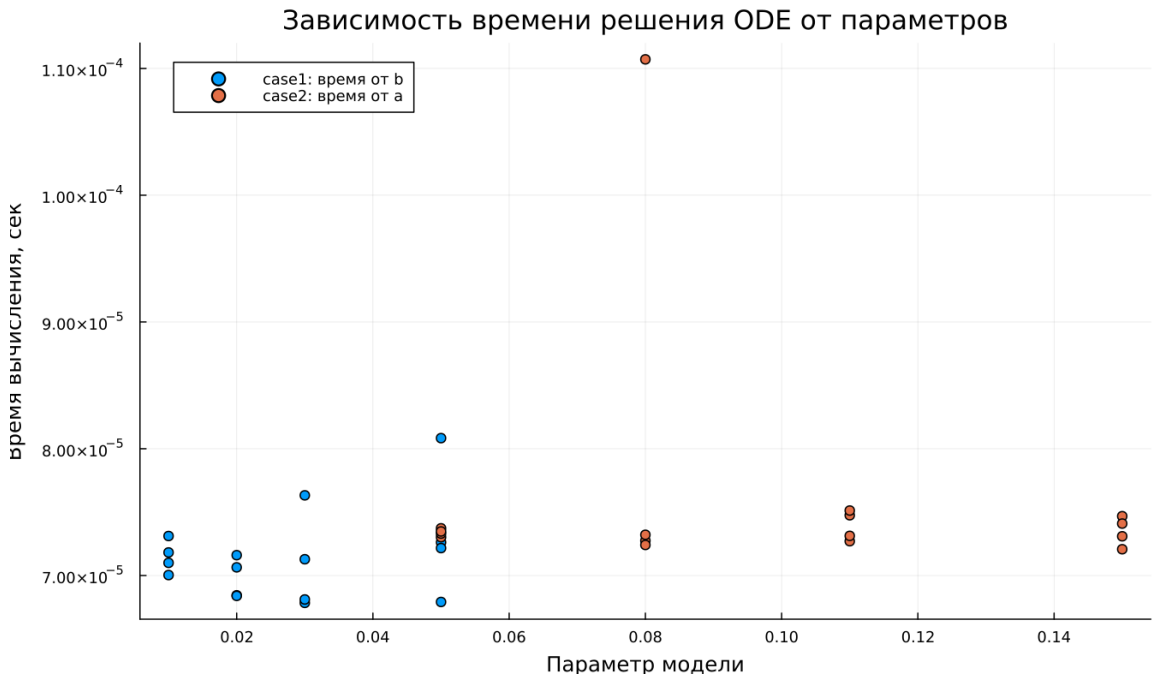

Рисунок 5.11: Зависимость времени вычисления

Результаты бенчмаркинга показывают, что время численного решения во всех проведённых экспериментах остаётся очень малым и имеет один порядок величины, примерно $\sim 10^{-4}$ секунд.

Для обеих моделей изменение параметров практически не влияет на продолжительность вычислений. Небольшие колебания времени можно объяснить особенностями используемого численного метода, а также работой адаптивного шага интегрирования.

Можно сделать следующие выводы:

- обе модели эффективно решаются численными методами;
- изменение параметров почти не влияет на вычислительную сложность;
- даже при экспоненциальном росте решения в case1 метод сохраняет малое время расчёта.

5.24 Выводы

1. Первая модель (case1) показывает неограниченный экспоненциальный рост числа инфицированных при неизменном числе восприимчивых особей. Это указывает на нефизичность модели и отсутствие механизма стабилизации.
2. Вторая модель (case2) воспроизводит характерную эпидемическую динамику: сначала число инфицированных растёт, затем достигает максимума и постепенно уменьшается.
3. Фазовые портреты подтверждают различие между моделями: в первой модели траектория вырождается в вертикальную линию, а во второй формируется полноценная кривая, соответствующая развитию эпидемии.
4. Параметры a и b оказывают существенное влияние на поведение системы: параметр a связан со скоростью уменьшения числа восприимчивых, а параметр b влияет на интенсивность изменения числа инфицированных.
5. Метрика norm_final позволяет количественно различать две модели: в первой модели она быстро увеличивается, а во второй отражает переход системы к конечному состоянию.
6. Максимальное число инфицированных I_{max} в первой модели возрастает без ограничения, тогда как во второй модели оно остаётся конечным и зависит от выбранных параметров.
7. Численное моделирование обеих систем выполняется быстро, а изменение параметров не приводит к существенному росту времени вычислений.

Список литературы

1. Конструирование эпидемиологических моделей
2. Зараза, гостья наша